Gomory cuts for integer optimization

Michel Bierlaire

Algorithm and examples

[1]:
```
import numpy as np
```

The cutting plane algorithm for integer optimization relies on the simplex algorithm. For the notebook to be self-contained, we include the code of the algorithm below, so that it can be called by the Gomory cut algorithm.

1 Simplex algorithm

[2]:
```
def simplexTableau(tableau):
    """
    :param tableau: the first simplex tableau
    :type tableau: pandas dataframe

    :return: p, q, opt, bounded  where
                - p is the column of the variable that must enter the basis, or␣
↪None,
                - q is the row of the variable that must leave the basis, or None,
                - opt is True if the tableau is optimal (in this case, p and q␣
↪are None)
                - bounded is True if basic direction is unbounded (in this case,␣
↪p and q are None)
    :rtype: int, int, bool, bool
    """
    mtab, ntab = tableau.shape
    m = mtab - 1
    n = ntab - 1

    reducedCost = tableau[-1, : -1]
    # Identify the negative reduced costs

    negativeReducedCost = reducedCost < -np.finfo(float).eps
    if not negativeReducedCost.any():
        # The tableau is optimal
        return None, None, True, True
```

```
    # In Python, True is larger than False. The next statement returns the
    # index of a True entry in the array, that is the index of a negative reduced cost.
    # It is the index of the variable that will enter the basis.
    p = np.argmax(negativeReducedCost)

    # Calculate the maximum step that can be done along the basic direction d[p]
    xb = tableau[:-1, -1]
    minusd = tableau[:-1, p]
    steps = np.array([xb[k] / minusd[k] if minusd[k] > 0 else np.inf for k in range(m)])
    q = np.argmin(steps)
    step = steps[q]
    if step == np.inf:
        # The tableau is unbounded
        return None, None, False, False
    else:
        return p, q, False, True
```

[3]:
```
def pivoting(tableau, p, q):
    """
    :param tableau: valid simplex tableau
    :type tableau: numpy.array 2D

    :param p: columns of the pivot
    :type p: int

    :param q: row of the pivot
    :type q: int

    :return: pivoted tableau
    :rtype: numpy.array 2D
    """
    m, n = tableau.shape
    if q >= m:
        raise Exception(f'The row of the pivot ({q}) must be between 0 and {m-1})')
    if p >= n:
        raise Exception(f'The column of the pivot ({p}) must be between 0 and {n - 1})')
    thepivot = tableau[q][p]
    if np.abs(thepivot) < np.finfo(float).eps:
        print(f'Tableau: {tableau})')
        print(f'Row {q} Col {p}')
        raise Exception(f'The pivot is too close to zero: {thepivot}')
    thepivotrow = tableau[q, :]
    newtableau = np.empty(tableau.shape)
```

```
    newtableau[q, :] = tableau[q, :] / thepivot
    for i in set(range(m)) - {q}:
        newtableau[i, :] = tableau[i, :] - tableau[i][p] * thepivotrow / thepivot
    return newtableau
```

[4]:
```
def prettyTableau(tableau):
    m, n = tableau.shape
    s = ''
    for i in range(m - 1):
        formattedRow = ['{:6.2g}' for k in tableau[i, :-1]]
        s += '\t'.join(formattedRow).format(*tableau[i, :-1])
        s += f'|{tableau[i, -1]:6.2f}'
        s += '\n'
    for i in range(n):
        s += '------\t'
    s += '\n'
    formattedRow = ['{:6.2g}' for k in tableau[m - 1, :-1]]
    s += '\t'.join(formattedRow).format(*tableau[m - 1, :-1])
    s += f'|{tableau[m - 1,- 1]:6.2f}'
    s += '\n'

    return s
```

[5]:
```
def simplexAlgorithmTableau(tableau):
    """
    :param tableau: valid simplex tableau
    :type tableau: numpy.array 2D

    :return: tableau, optimal, unbounded, where tableau is the tableau from the␣
 ↪last iteration,
                                          optimal is True if the last tableau is␣
 ↪optimal,
                                          unbounded is True if the problem is␣
 ↪unbounded.
    :rtype: numpy.array 2D, bool, bool
    """
    while True:
        colPivot, rowPivot, optimal, bounded = simplexTableau(tableau)
        if optimal:
            return tableau, True, False
        if not bounded:
            return tableau, False, True
        tableau = pivoting(tableau, colPivot, rowPivot)
```

[6]:
```
def getRow(tableau, index):
    """
    :param tableau: a valid simplex tableau.
```

```
    :type tableau: numpy.array 2D

    :param index: the index of the variable
    :type index: int

    :return: row index if the variable is basic, None otherwise
    :rtype: int
    """
    mtab, ntab = tableau.shape
    m = mtab - 1
    n = ntab - 1
    if index not in range(n):
         raise Exception(f'The index of the variable ({index}) must be between 0␣
↪and {n-1})')
    column = tableau[:, index]
    rowIndex = None
    for j in range(mtab):
        if np.abs(column[j]) > np.sqrt(np.finfo(float).eps):
            # The entry is non zero
            if rowIndex is None and np.abs(column[j] - 1) <= np.finfo(float).eps:
                # The entry is one, and the index has not been found yet.
                rowIndex = j
            else:
                return None
    return rowIndex
```

[7]:
```
def simplex(A, b, c):
    """
    :param A: m x n matrix of the constraints
    :type A: numpy.array 2D

    :param b: m vector, left-hand side of the constraints.
    :type b: numpy.array 1D

    :param c: n vector, coefficients of the objective function
    :type c: numpy.array 1D

    :return: tableau, unbounded, infeasible
                where  - tableau is the tableau from the last iteration,
                       - unbounded is True if the problem is unbounded,
                       - infeasible is True if the problem is infeasible,
    :rtype: numpy.array 2D, bool, bool
    """

    m, n = A.shape
    if b.shape[0] != m:
```

```
        raise Exception(f'Incompatible sizes: A is {m}x{n}, b is of length {b.
↪shape[0]}, and should be {m}')
    if c.shape[0] != n:
        raise Exception(f'Incompatible sizes: A is {m}x{n}, c is of length {c.
↪shape[0]}, and should be {n}')

    # All elements of b must be non negative.
    negativeb = np.argwhere(b < 0)
    for i in negativeb:
        A[i, :] = - A[i, :]
        b[i] = - b[i]

    # First tableau for the auxiliary problem
    tableau = np.empty((m + 1, n + m + 1))
    tableau[:m, :n] = A
    tableau[:m, n:n + m] = np.eye(m)
    tableau[:m, n + m:n + m + 1] = b.reshape(m, 1)
    # The last row
    tableau[-1, :n] = np.array([-np.sum(tableau[:m, k]) for k in range(n)]).
↪copy()
    tableau[-1, n:n + m] = 0.0
    tableau[-1, -1] = -np.sum(tableau[:m, -1])


    # Solve the auxiliary problem
    phaseOneTableau, optimal, unbounded = simplexAlgorithmTableau(tableau)

    if unbounded:
        return tableau, True, False

    if phaseOneTableau[-1, -1]  < -np.sqrt(np.finfo(float).eps):
        # Infeasible problem
        return phaseOneTableau, False, True

    # Remove the auxiliary variables from the basis
    clean = False
    rowsToRemove = []
    basicRows = np.array([getRow(phaseOneTableau,k) for k in range(m + n)])

    while not clean:
        basicIndices = set(np.where(basicRows != None)[0])
        # Check if some auxiliary variables are in the basis
        tobeCleaned = set(basicIndices).intersection(set(range(n,n+m)))
        if tobeCleaned == set():
            clean = True
        else:
            auxiliaryColumn = tobeCleaned.pop()
```

```
            rowpivotIndex = basicRows[auxiliaryColumn]
            rowpivot = phaseOneTableau[rowpivotIndex,:]
            originalNonbasic = list(set(range(n)) - set(basicIndices))
            nonzerosPivots = abs(rowpivot[originalNonbasic]) > np.finfo(float).
↪eps
            if nonzerosPivots.any():
                # It is possible to pivot
                colpivot = originalNonbasic[np.argmax(nonzerosPivots)]
                phaseOneTableau = pivoting(phaseOneTableau, colpivot,␣
↪rowpivotIndex)
                basicRows[colpivot] = rowpivotIndex
                basicRows[auxiliaryColumn] = None
            else:
                # It is not possible to pivot. There is a redundant
                # constraint to be removed.
                rowsToRemove.append(rowpivotIndex)
                phaseOneTableau = np.delete(phaseOneTableau,rowsToRemove,0)
                basicRows = np.array([getRow(phaseOneTableau,k) for k in range(m␣
↪+ n)])

    # Delete columns
    startPhaseTwo = np.delete(phaseOneTableau, range(n, n + m), 1)
    m -= len(rowsToRemove)
    basicRows = np.array([getRow(startPhaseTwo, k) for k in range(n)])

    # Calculate last row

    basicIndices = list(np.where(basicRows != None)[0])
    nonbasicIndices = list(np.where(basicRows == None)[0])
    cb = c[basicIndices]
    for k in nonbasicIndices:
        startPhaseTwo[-1, k] = c[k] - np.array([c[j] *␣
↪startPhaseTwo[basicRows[j], k] for j in basicIndices]).sum()
    startPhaseTwo[-1, -1] = - np.array([c[j] * startPhaseTwo[basicRows[j], -1]␣
↪for j in basicIndices]).sum()

    # Phase II

    phaseTwoTableau, optimal, unbounded = simplexAlgorithmTableau(startPhaseTwo)
    return phaseTwoTableau, unbounded, False
```

[8]:
```
def getResults(finalTableau):
    mtab, ntab = finalTableau.shape
    m = mtab - 1
    n = ntab - 1
    basicRows = [getRow(finalTableau, k) for k in range(n)]
```

```
    solution = [float(finalTableau[basicRows[k], -1]) if basicRows[k] is not␣
↪None else 0 for k in range(n)]
    copt = -finalTableau[-1, -1]
    return solution, copt
```

[9]:
```
def printResults(res):
    finalTableau, unbounded, infeasible = res
    if unbounded:
        print('Unbounded problem')
        return None, None
    elif infeasible:
        print('Infeasible problem')
        return None, None
    else:
        print(prettyTableau(finalTableau))
        solution, copt = getResults(finalTableau)
        print(f'Optimal solution: {solution}')
        print(f'Optimal value: {copt}')
        return solution, copt
```

2 Gomory cut

This is Algorithm 26.3 in Bierlaire (2015) Optimization: principles and algorithms, EPFL Press.

The following function tests if a real number is actually an integer.

[10]:
```
def isInteger(x):
    # We identify a solution as integer if it does not deviate from its
    # rounded version by the square root of the machine epsilon.
    return np.abs(x - np.round(x)) <= np.sqrt(np.finfo(float).eps)
```

The following function takes an optimization problem as input, identifies all Gomory cuts and returns an updated problem and an optimal tableau.

[11]:
```
def gomory(A, b, c):
    """This function solves a linear optimization problem,
        identifies the Gomory cuts, and generates the data of a new
        problem, including the cuts.

    :param A: matrix A of the constraints
    :type A: numpy.array 2D

    :param b: vector right-hand side of the constraint
    :type b: numpy.array 1D

    :param c: cost vector of the objective function
    :type c: numpy.array 1D
```

```
    :return: nGomory, newA, newb, newc, tableau where

         - nGomory, is the number of cuts that have been introduced,
         - newA, newb, newc is the data of the new optimization problem,
         - tableau is the optimal tableau of the optimization problem.

    :rtype: int, numpy.array 2D, numpy.array 1D, numpy.array 1D, numpy.array 2D

    """
    m, n = A.shape
    print('==================================')
    print(f'Problem with {n} variables and {m} constraints')
    tableau, unbounded, infeasible = simplex(A, b, c)
    solution, copt = getResults(tableau)
    print(f'Solution: {solution} [{copt}]')
    if unbounded:
        print('Unbounded problem')
        return 0, None, None, None, None
    if infeasible:
        print('Infeasible')
        return 0, None, None, None, None
    lastColumn = tableau[:, -1]
    fractionalRows = np.nonzero(np.logical_not(isInteger(lastColumn[:-1])))[0]
    # Additional rows for the Gomory cut
    nGomory = len(fractionalRows)
    gamma = np.floor(tableau[fractionalRows, :-1])
    bplus = np.floor(tableau[fractionalRows, -1])
    # Build the new matrices and vectors
    newA = np.concatenate((A, np.zeros((m, nGomory))), 1)
    newRows = np.concatenate((gamma, np.eye(nGomory)), 1)
    newA = np.concatenate((newA, newRows))
    newb = np.concatenate((b, bplus))
    newc = np.concatenate((c, np.zeros((nGomory))))
    print(f'Number of Gomory cuts: {nGomory}')
    return nGomory, newA, newb, newc, tableau
```

We keep on adding Gomory cuts until we obtain an integer solution.

[12]:
```
def gomoryAlgorithm(A, b, c):
    nGomory = None
    while nGomory != 0:
        nGomory, A, b, c, tableau = gomory(A, b, c)
    return tableau
```

3 Example 1

$$\min x_1 - 2x_2$$

subject to

$$\begin{aligned} -4x_1 + 6x_2 &\leq 5, \\ x_1 + x_2 &\leq 5, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[13]:
```
A = np.array([[-4, 6, 1, 0], [1, 1, 0, 1]])
b = np.array([5, 5])
c = np.array([1, -2, 0, 0])
```

[14]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [2.5000000000000004, 2.5, 0, 0] [-2.4999999999999996]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [1.7499999999999998, 2.0, 0, 1.2500000000000009, 0,
0.2500000000000002] [-2.25]
Number of Gomory cuts: 3
==================================
Problem with 9 variables and 7 constraints
Solution: [1.9999999999999964, 1.9999999999999978, 0.9999999999999991,
1.0000000000000067, 2.220446049250309e-15, 1.0000000000000022, 0, 0,
2.2204460492502934e-16] [-1.9999999999999991]
Number of Gomory cuts: 0
```

[15]:
```
solution, copt
```

[15]:
```
([1.9999999999999964,
  1.9999999999999978,
  0.9999999999999991,
  1.0000000000000067,
  2.220446049250309e-15,
  1.0000000000000022,
  0,
  0,
  2.2204460492502934e-16],
 -1.9999999999999991)
```

Solution: $x_1 = 2$, $x_2 = 2$. Optimal value: -2.

4 Example 2

$$\min -3x_1 - 13x_2$$

subject to

$$\begin{aligned} 2x_1 + 9x_2 &\leq 29, \\ 11x_1 - 8x_2 &\leq 79, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[16]:
```
A = np.array([[2, 9, 1, 0], [11, -8, 0, 1]])
b = np.array([29, 79])
c = np.array([-3, -13, 0, 0])
```

[17]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [8.2, 1.4, 0, 0] [-42.8]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [8.0, 1.4444444444444438, 0, 2.555555555555546, 2.111111111111102, 0]
[-42.77777777777777]
Number of Gomory cuts: 3
==================================
Problem with 9 variables and 7 constraints
Solution: [5.500000000000092, 1.9999999999999782, 0, 34.499999999998806,
33.49999999999882, 2.499999999999908, 0.9999999999999778, 1.4999999999999294, 0]
[-42.5]
Number of Gomory cuts: 5
==================================
Problem with 14 variables and 12 constraints
Solution: [1.0000000000074172, 2.999999999998162, 1.5045742429720344e-12,
16.000000000000256, 89.99999999990588, 6.999999999992596, 2.9999999999980336,
4.999999999994401, 0.9999999999997475, 0, 0, 0, 0, 89.9999999999074]
[-41.99999999999858]
Number of Gomory cuts: 0
```

[18]:
```
solution, copt
```

[18]:
```
([1.0000000000074172,
  2.999999999998162,
  1.5045742429720344e-12,
  16.000000000000256,
  89.99999999990588,
  6.999999999992596,
  2.9999999999980336,
```

```
 4.999999999994401,
 0.9999999999997475,
 0,
 0,
 0,
 0,
 89.9999999999074],
-41.99999999999858)
```

Solution: $x_1 = 1$, $x_2 = 3$. Optimal value: -42.

5 Example 3

$$\min -13x_1 - 8x_2$$

subject to

$$\begin{aligned} x_1 + 2x_2 &\leq 10, \\ 5x_1 + 2x_2 &\leq 20, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[19]:
```
A = np.array([[1, 2, 1, 0], [5, 2, 0, 1]])
b = np.array([10, 20])
c = np.array([-13, -8, 0, 0])
```

[20]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [2.5, 3.75, 0, 0] [-62.5]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [2.2857142857142856, 3.8571428571428577, 0, 0.8571428571428574, 0,
3.571428571428572] [-60.57142857142858]
Number of Gomory cuts: 4
==================================
Problem with 10 variables and 8 constraints
Solution: [3.0, 2.4999999999999982, 2.000000000000002, 0, 0.5000000000000004,
3.5000000000000013, 1.0000000000000036, 1.0000000000000036, 0,
1.0000000000000027] [-58.99999999999999]
Number of Gomory cuts: 3
==================================
Problem with 13 variables and 11 constraints
Solution: [2.0, 3.9999999999999996, 2.6645352591003757e-15, 2.0,
1.0000000000000018, 4.000000000000001, 0, 0.0, 6.661338147750939e-16,
8.881784197001252e-16, 1.0000000000000004, 1.0000000000000009, 0]
```

```
[-57.99999999999999]
Number of Gomory cuts: 0
```

[21]:
```
solution, copt
```

[21]:
```
([2.0,
  3.9999999999999996,
  2.6645352591003757e-15,
  2.0,
  1.0000000000000018,
  4.000000000000001,
  0,
  0.0,
  6.661338147750939e-16,
  8.881784197001252e-16,
  1.0000000000000004,
  1.0000000000000009,
  0],
 -57.99999999999999)
```

Solution: $x_1 = 2$, $x_2 = 4$. Optimal value: -58.

6 Example 4

$$\min -4x_1 + 6x_2$$

subject to

$$\begin{aligned} -4x_1 + 6x_2 &\leq 5, \\ x_1 + x_2 &\leq 5, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[22]:
```
A = np.array([[-4, 6, 1, 0], [1, 1, 0, 1]])
b = np.array([5, 5])
c = np.array([1, -2, 0, 0])
```

[23]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [2.5000000000000004, 2.5, 0, 0] [-2.4999999999999996]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [1.7499999999999998, 2.0, 0, 1.2500000000000009, 0,
0.2500000000000002] [-2.25]
Number of Gomory cuts: 3
==================================
```

```
Problem with 9 variables and 7 constraints
Solution: [1.9999999999999964, 1.9999999999999978, 0.9999999999999991,
1.0000000000000067, 2.220446049250309e-15, 1.0000000000000022, 0, 0,
2.2204460492502934e-16] [-1.9999999999999991]
Number of Gomory cuts: 0
```

[24]:
```
solution, copt
```

[24]:
```
([1.9999999999999964,
  1.9999999999999978,
  0.9999999999999991,
  1.0000000000000067,
  2.220446049250309e-15,
  1.0000000000000022,
  0,
  0,
  2.2204460492502934e-16],
 -1.9999999999999991)
```

Solution: $x_1 = 2$, $x_2 = 2$. Optimal value: -2.

7 Example 5

$$\min 2x_1 + 3x_2$$

subject to

$$\begin{aligned} 2x_1 + x_2 &\geq 6, \\ x_1 + 3x_2 &\geq 7, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[25]:
```
A = np.array([[2, 1, -1, 0], [1, 3, 0, -1]])
b = np.array([6, 7])
c = np.array([2, 3, 0, 0])
```

[26]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [2.2, 1.6, 0, 0] [9.200000000000001]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [2.0, 2.0, 0, 1.0, 0.0, 0] [10.0]
Number of Gomory cuts: 0
```

[27]:
```
solution, copt
```

[27]:
```
([2.0, 2.0, 0, 1.0, 0.0, 0], 10.0)
```

Solution: $x_1 = 2$, $x_2 = 2$. Optimal value: 10.

8 Example 6

$$\max 8x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 + x_2 &\leq 6, \\ 9x_1 + 5x_2 &\leq 45, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

[28]:
```
A = np.array([[1, 1, 1, 0], [9, 5, 0, 1]])
b = np.array([6, 45])
c = np.array([-8, -5, 0, 0])
```

[29]:
```
t = gomoryAlgorithm(A, b, c)
solution, copt = getResults(t)
```

```
==================================
Problem with 4 variables and 2 constraints
Solution: [3.75, 2.25, 0, 0] [-41.25]
Number of Gomory cuts: 2
==================================
Problem with 6 variables and 4 constraints
Solution: [5.0, 0.0, 1.0, 0, 0.0, 0] [-40.0]
Number of Gomory cuts: 0
```

[30]:
```
solution, copt
```

[30]:
```
([5.0, 0.0, 1.0, 0, 0.0, 0], -40.0)
```

Solution: $x_1 = 5$, $x_2 = 0$. Optimal value: -40 for minimization, 40 for maximization.